SE 801: Project

Technical Document

Project Name: FleetTrain

Submitted by

Nazmus Sakib- BSSE 1225

Supervisor

Dr B M Mainul Hossain

Professor

Institute of Information Technology, University of Dhaka

Date of Submission

01 December, 2024



Letter of Transmittal

01 December, 2024
BSSE 4th Year Exam Committee
Institute of Information Technology
University of Dhaka

Subject: Technical Report Submission - **FleetTrain**.

Sir,
I am submitting the technical report for our Software Project Lab 3, "**FleetTrain**".
This report comprehensively covers the project's technical details, including design, implementation, and testing phases. While I've tried to do my best, I welcome your feedback for improvement.

Thank you for your consideration.

Sincerely,

Nazmus Sakib
BSSE-1225
Institute of Information Technology
University of Dhaka



Supervisor's Signature

Acknowledgement

I am grateful to the Institute of Information Technology for giving me the opportunity to work on the project "FleetTrain". I would like to convey my tremendous gratitude to my supervisor, Dr. B M Mainul Hossain, Professor, Institute of Information Technology, University of Dhaka, for providing me with guidance and support throughout the project. His expertise and insights were instrumental in shaping the direction of the project. Lastly, I would like to thank my classmates. They have always been helpful and provided valuable insights from time to time.

Abstract

FleetTrain is a cloud-based platform designed to enable scalable and privacy-preserving distributed machine learning through federated learning. With the growing demand for AI solutions and stringent data privacy regulations like GDPR, traditional centralized training approaches are increasingly inadequate. FleetTrain addresses these challenges by allowing users to collaboratively train machine learning models on decentralized datasets without compromising data security. The platform combines a Jupyter-like notebook interface with a robust federated learning engine that distributes training tasks across multiple devices or cloud instances, aggregating results into a global model while keeping sensitive data local. Key features include real-time monitoring, task distribution, secure user management, and support for popular ML frameworks like TensorFlow and PyTorch. By leveraging federated learning principles, FleetTrain democratizes access to advanced machine learning capabilities, enabling researchers, developers, and organizations with limited infrastructure to participate in collaborative model training. The platform aims to foster innovation in AI while ensuring the highest standards of data privacy, security, and scalability, making it an ideal solution for sensitive data environments.

Table of Contents

Introduction **1**
Background 1
Problem Statement 1
Proposed Solution 2
Project Timeline 3
Purpose of Document 4
Quality Function Deployment (QFD) **5**
Expected Requirements 5
Normal Requirements 5
Exciting Requirements 6
Usage Scenario **7**
Scenario Based Modeling **8**
Use Case Diagram 8
Level 0: FleetTrain 8
Level 1: FleetTrain 9
Level 1.1: Account Management 9
Level 1.2: Notebook Management 10
Level 1.3: Notebook Execution Environment 10
Level 1.4: Resource Management 11
Data Based Modeling **12**
Class-Based Modeling **14**
Analysis Classes 14
Class Cards 15
Developer Class 15
Admin Class 16
Notebook Class 16
Resource Class 17
Dataset Class 17
ResourceManager Class 18
Orchestrator Class 18
Class Responsibility Collaboration (CRC) 19

Architectural Design **20**
Architectural Context Diagram 20
Top Level Components 21
Define Archetypes 21
Refine the Architecture into Components 24
Instantiations of the system 25
Deployment Diagram 27
Preliminary Test Plan **28**
High Level Description of Testing Goals 28
Test Cases 28
Conclusion **30**
References **30**

List of Figures

Figure-1: Project Timeline (Estimated) 3
Figure-2: Level 0 - FleetTrain 8
Figure-3: UseCase Level 1 (Modules of FleetTrain) 9
Figure-4: Level 1.1 - Account Management 9
Figure-5: Level 1.2 - Notebook Management 10
Figure-6: Level 1.3 - Notebook Execution Environment 10
Figure-7: Level 1.4 - Resource Management 11
Figure-8: ER-Diagram of FleetTrain 12
Figure-9: CRC Diagram of FleetTrain 19
Figure-10: Architectural Context Diagram 20
Figure-11: Top Level Components of FleetTrain 21
Figure-12: MVC Pattern 22
Figure-13: Event Driven Architecture 22
Figure-14: Client-Server Architecture 23
Figure-15: Master-Worker Architecture 24
Figure-16: Notebook Management with MVC Pattern 25
Figure-17: Resource Allocation with Event Driven Architectural Pattern 25
Figure-18: Notebook Execution with Client-Server Architecture 26
Figure-19: Orchestration with Master Worker Architectural Pattern 26
Figure-20: Deployment Diagram for FleetTrain 27

List of Figures

Table-1: Schema of FleetTrain 13
Table-2: Developer Class Card 15
Table-3: Admin Class Card 16
Table-4: Notebook Class Card 16
Table-5: Resource Class Card 17
Table-6: Dataset Class Card 17
Table-7: ResourceManager Class Card 18
Table-8: Orchestrator Class Card 18
Table-9: Architecture to Component Refinement 24

Introduction

FleetTrain is a cloud-based platform designed to facilitate distributed machine learning through federated learning techniques. It is designed to simplify and optimize the process of running code in a distributed and scalable manner. It enables multiple devices to collaboratively train machine learning models without centralizing sensitive data, thereby enhancing privacy and scalability in AI applications. Inspired by platforms like Google Colab and systems like Apache Hadoop, FleetTrain empowers users to execute code across multiple devices, manage notebooks, handle datasets, and allocate resources efficiently. By integrating distributed computing capabilities with a central server-client architecture, FleetTrain provides a versatile solution for collaborative work, large-scale data processing, and secure federated machine learning (FML).

Background

In recent years, the explosion of data and advancements in artificial intelligence (AI) have paved the way for powerful machine learning models across industries such as healthcare, finance, and IoT. With the rapid growth of data-driven applications, especially in machine learning and artificial intelligence, the demand for platforms that can handle large-scale computations efficiently has increased significantly. Traditional methods of distributed computing often require substantial setup, management of compute clusters, and complex configurations, which can be a barrier for individual researchers and small organizations. Additionally, privacy concerns and regulations around data sharing have led to the adoption of federated learning, where model training happens across decentralized datasets, allowing data to remain on local devices. However, implementing federated learning can be complex without a well-orchestrated platform that handles data privacy, distributed task execution, and model aggregation. To address this gap, FleetTrain aims to simplify federated learning by providing a cloud-based platform that offers a secure, scalable, and accessible solution for distributed machine learning.

Problem Statement

Organizations seeking to leverage federated learning often face multiple challenges, including:

- Data Privacy and Compliance: Sensitive data, especially in sectors like healthcare and finance, cannot be centralized due to privacy regulations such as GDPR and HIPAA. Therefore, it is challenging to develop AI models without compromising data privacy.

- Infrastructure Complexity: Deploying and managing federated learning setups requires significant expertise in distributed systems, model synchronization, and privacy-preserving techniques, which can be resource-intensive and error-prone.

- Scalability and Resource Management: Efficiently scaling federated learning across multiple devices with varying computational capacities is challenging. It requires robust coordination and resource management to ensure that training is balanced across devices and that models converge effectively.

- Accessibility for Data Scientists: Data scientists often lack the tools needed to experiment with federated learning in a streamlined environment. The complexity of setting up a federated learning system can delay innovation and limit the accessibility of this approach.

Existing platforms for distributed computing and collaborative notebook management often fall short in providing a seamless and secure environment for distributed, privacy-conscious computation. For users who need to process large datasets across multiple devices, managing resources manually and ensuring data privacy pose significant challenges. Furthermore, implementing federated learning workflows is difficult without a system that can handle decentralized data storage, model training, and secure aggregation. These limitations highlight the need for a platform that combines the flexibility of notebook-based development with robust distributed computing capabilities and privacy-preserving features.

Proposed Solution

FleetTrain is a cloud-based, privacy-preserving platform designed to make federated learning accessible and manageable. FleetTrain addresses these challenges by providing a centralized platform that orchestrates code execution and resource management across client devices in a distributed manner. The proposed solution involves a client-server model, where a central server coordinates task distribution and aggregation of results. It leverages distributed computing principles to allow data scientists to securely train machine learning models across multiple devices without centralizing data. Key aspects of FleetTrain's solution include:

- Notebook Management: A user-friendly interface for creating, organizing, and executing notebooks.

- Distributed Computing: A centralized server that manages code execution across client devices, leveraging each device’s resources.

- Federated Learning Support: Privacy-preserving model aggregation, allowing model training across decentralized data while protecting user data.

- Resource Management: Efficient allocation and monitoring of resources like CPU and memory across devices to optimize performance.
- Efficient Model Aggregation and Scalability: The platform facilitates real-time model aggregation, combining updates from participating devices and producing a unified model.

- Containerized Environment: FleetTrain uses Docker to provide a consistent, reproducible environment, enabling data scientists to focus on modeling without infrastructure concerns.

- Real-time Monitoring and Analytics: FleetTrain offers real-time insights into model training, device health, and system performance, ensuring a smooth and transparent process.

By simplifying the federated learning process, FleetTrain empowers organizations to adopt privacy-preserving machine learning solutions that respect data sovereignty, are accessible to data scientists, and are scalable to meet the demands of modern AI applications.

Project Timeline

The estimated timeline of FleetTrain is given below:

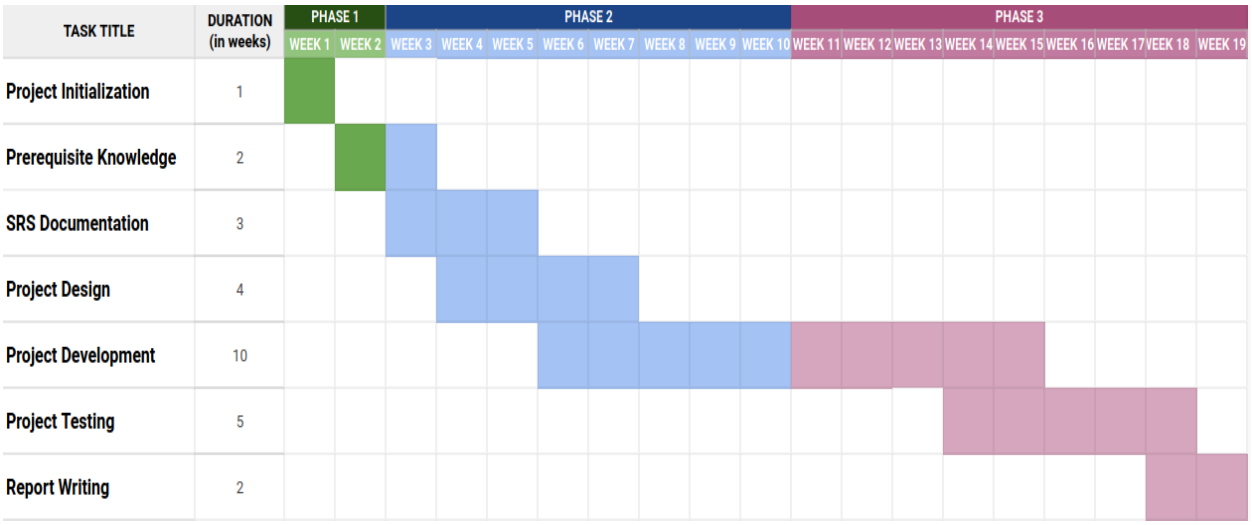


Figure-1: Project Timeline (Estimated)

Purpose of Document

The purposes of this document are-

- Identify and analyze the requirements
- Improve understanding
- Design the architecture
- Reduce the development effort
- Design the test plan

Quality Function Deployment (QFD)

The Quality Function Deployment (QFD) is a process and collection of tools that is used to precisely identify customer needs and convert them into comprehensive engineering specifications and production plans for the solutions that meet those requirements. These explicit and implicit client needs or demands are referred to as the "voice of the customer." In order to drive customer needs (or VOC) from the assembly level down through the sub-assembly, component, and production process levels, it is necessary to apply quality function design (QFD). A defined set of matrices is provided by the QFD approach and is used to aid this process. QFD identifies three types of requirements:

- Normal Requirements
- Expected Requirements
- Exciting Requirements

Expected Requirements

Expected requirements are fundamental to the system or product and may be so basic that the client does not state them explicitly. There will be significant dissatisfaction in their absence. The expected requirements of this project are-

- Basic Client-Server Communication: The client must connect to the central server reliably, enabling fundamental communication for task distribution and result aggregation.
- Notebook Management: Users should be able to create, edit, and save notebooks, making it easy to organize and manage their code within the platform.
- Task Distribution: The central server should be able to distribute tasks to connected client devices based on their availability and resource capacity.
- Basic Resource Monitoring: Track basic CPU and memory usage of each device involved in task execution to prevent overloading and allow efficient allocation.
- User Management: A secure user authentication and role-based access control to manage user permissions and facilitate collaboration must be implemented within FleetTrain.

Normal Requirements

Normal requirements are the goals and objectives that are discussed for a system or product at client meetings. The client is satisfied if these conditions are met. The normal requirements of this project are-

- Real-Time Monitoring of Task Progress: Users should be able to view the status and progress of tasks in real time from the central server.
- Federated Learning Support: Model training across decentralized datasets, allowing local model updates to be sent to the central server for aggregation should be implemented.

- Error Handling and Fault Tolerance: FleetTrain should be able to handle network interruptions and device unavailability, allowing tasks to be redistributed or retried as needed.
- Notebook Collaboration: Multiple users should be allowed to collaborate on notebooks, supporting concurrent editing and version control.

Exciting Requirements

These features go beyond the customer's expectations and prove to be very satisfying when present. The exciting requirements of this project are-

- Research-Informed Federated Learning Implementation: Latest research trends regarding federated learning will be followed for the system.
- Dynamic Resource Scaling: Automatically adjust resources based on task demand, enabling efficient load balancing and support for variable workloads.
- Compatibility with Popular ML Frameworks: Popular ML libraries (e.g., TensorFlow, PyTorch) can be used to simplify distributed training and federated learning tasks.

Usage Scenario

FleetTrain enables users to collaboratively train machine learning models across multiple devices in a federated learning environment. Users from different organizations or departments can participate without sharing sensitive data, making it ideal for scenarios requiring data privacy and distributed resources.Usage scenario for the platform can be segmented as follows:

Account Management

Users can create a new account through signing up using necessary information. After that users can sign in using email and password and use the system. They can also logout and edit account information.

Notebook Management

After signing in, users can create new notebooks. All previous notebooks will be listed as well and they can open any existing notebook for editing and executing. Besides these, users can delete and download notebooks as well.

Notebook Execution Environment

Inside a notebook, the user will see a classic notebook environment with features like add cell, delete cell and run cell. They can also upload datasets to the environment. There will be a configuration panel and users can select if they want to run it centrally or in a distributed way. The default setting is central. In case of distribution, the user will set which cell to run centrally and which to run in a distributed way. Rest of the features will be as usual and the system will handle the data distribution, federated training and model aggregation automatically.

Resource Management

Admins can add new resources, that is, devices to the list of existing resources. They can see which resources are being used by which user and which notebook is assigned to the resource in runtime. They can also delete any resource if it is corrupted.

Scenario Based Modeling

Scenario-based modeling is an approach used in software development and design to create models that depict how a system behaves or reacts in various real-world situations or scenarios. These scenarios are typically based on user interactions, events, or conditions and help developers understand and visualize how the software will function in different contexts. By simulating these scenarios, developers can better identify potential issues, make informed design decisions, and ensure that the software meets user requirements across different usage contexts.

Use Case Diagram

A use case diagram is a visual representation in software engineering that illustrates how a system interacts with its external entities or actors to achieve specific goals or tasks. It shows the relationships between use cases (representing system functionalities) and actors (representing external users or systems). Use case diagrams are typically created through requirements gathering, where system functionalities and user interactions are identified, and then diagrammed using specialized software modeling tools.

Level 0: FleetTrain

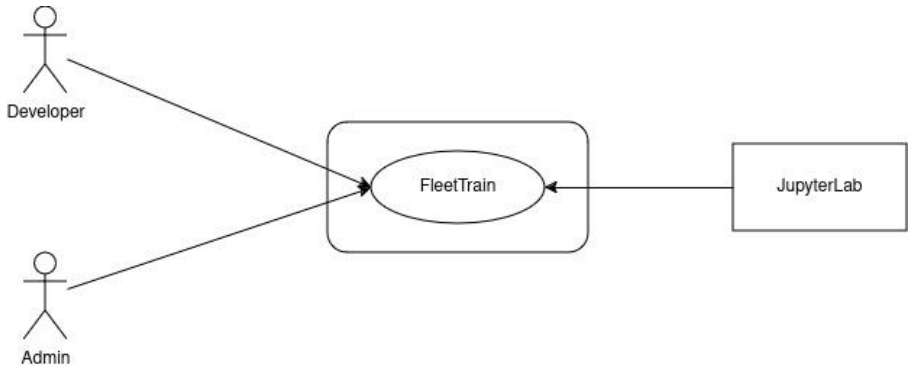


Figure-2: Level 0 - FleetTrain

Primary Actor: Developer, Admin

Secondary Actor: JupyterLab

Goal in Context: The diagram shown above represents the whole FleetTrain platform.

Level 1: FleetTrain

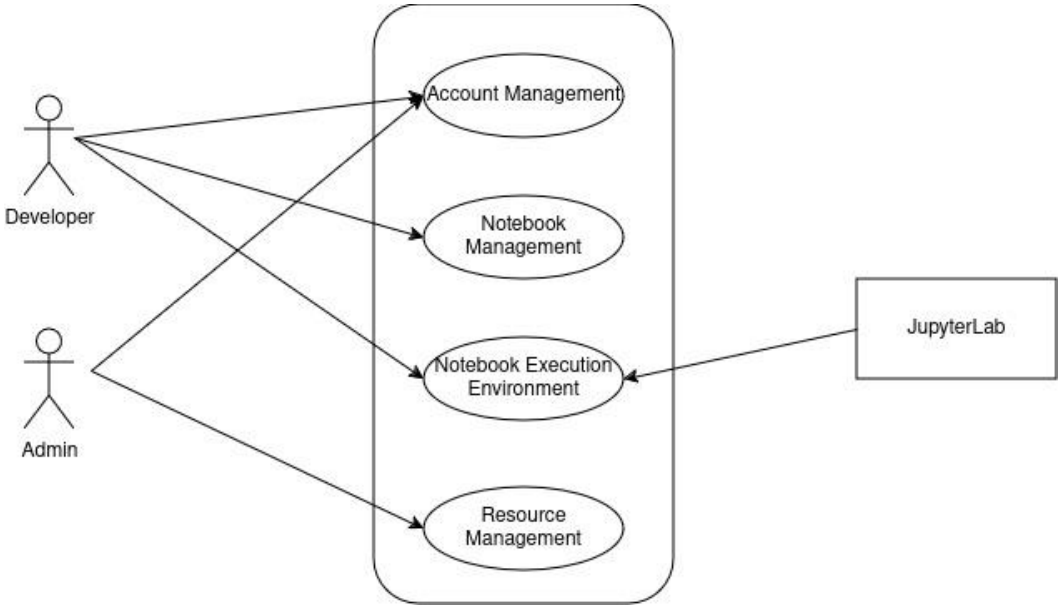


Figure-3: Level 1 - FleetTrain

Primary Actor: Developer, Admin

Secondary Actor: JupyterLab

Goal in Context: The diagram shows the modules of FleetTrain. FleetTrain consists of 4 modules. The modules can be elaborated as follows:

Level 1.1: Account Management

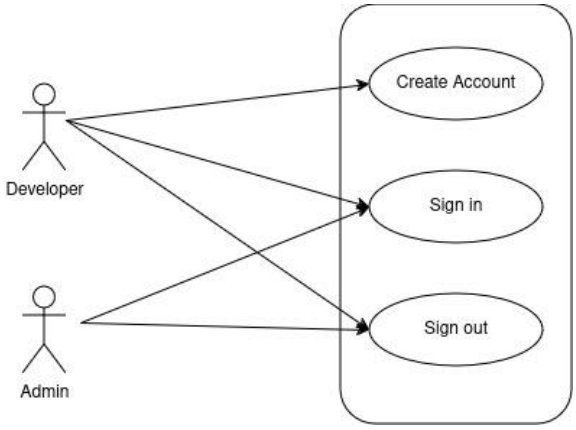


Figure-4: Level 1.1 - Account Management

Primary Actor: Developer, Admin

Goal in Context: The diagram shows the Account management of FleetTrain.

Level 1.2: Notebook Management

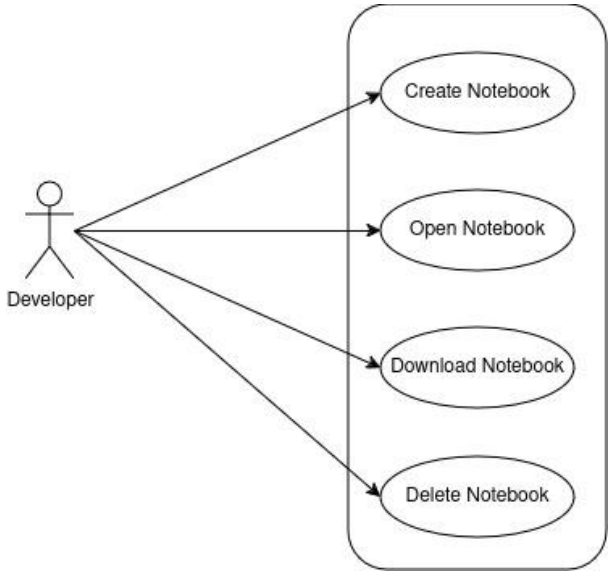


Figure-5: Level 1.2 - Notebook Management

Primary Actor: Developer

Goal in Context: The diagram shows the Notebook file management of FleetTrain.

Level 1.3: Notebook Execution Environment

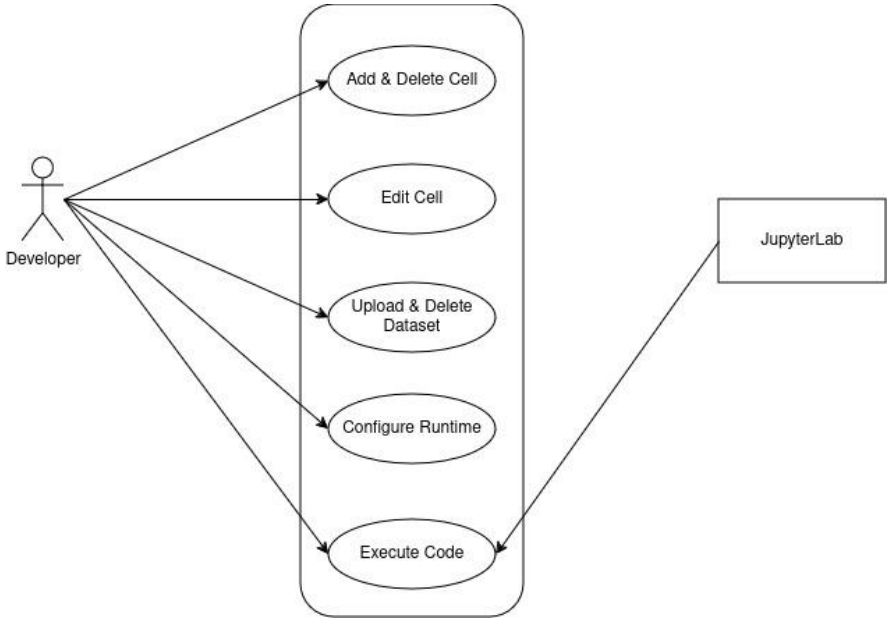


Figure-6: Level 1.3 - Notebook Execution Environment

Primary Actor: Developer

Secondary Actor: JupyterLab

Goal in Context: The diagram shows the Notebook execution of FleetTrain.

Level 1.4: Resource Management

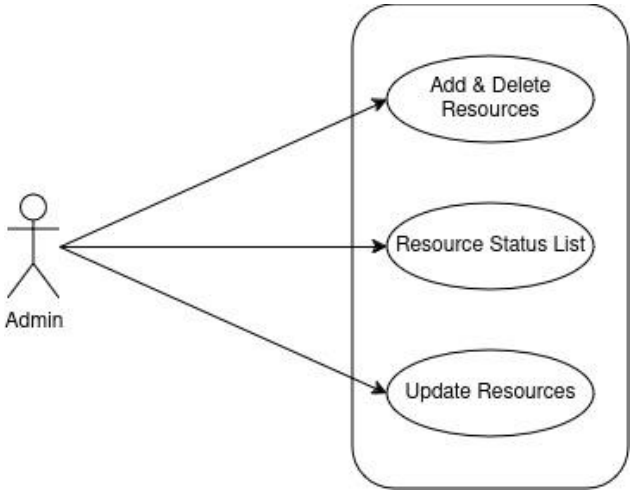


Figure-7: Level 1.4 - Resource Management

Primary Actor: Admin

Goal in Context: The diagram shows the Resource management of FleetTrain.

Data Based Modeling

Database modeling is a visual representation of a database structure and how data is organized within it. It involves creating entity-relationship diagrams (ERDs) that depict tables (entities), attributes (columns), and the relationships between them. Database modeling is typically achieved by analyzing the data requirements of a system, identifying entities and their attributes, and then using modeling tools or software to create ERDs that serve as a blueprint for database design and implementation.

The entities of FleetTrain after analyzing the usage story are:

- Notebook
- Resource
- User

The ER-Diagram of FleetTrain can be shown as below:

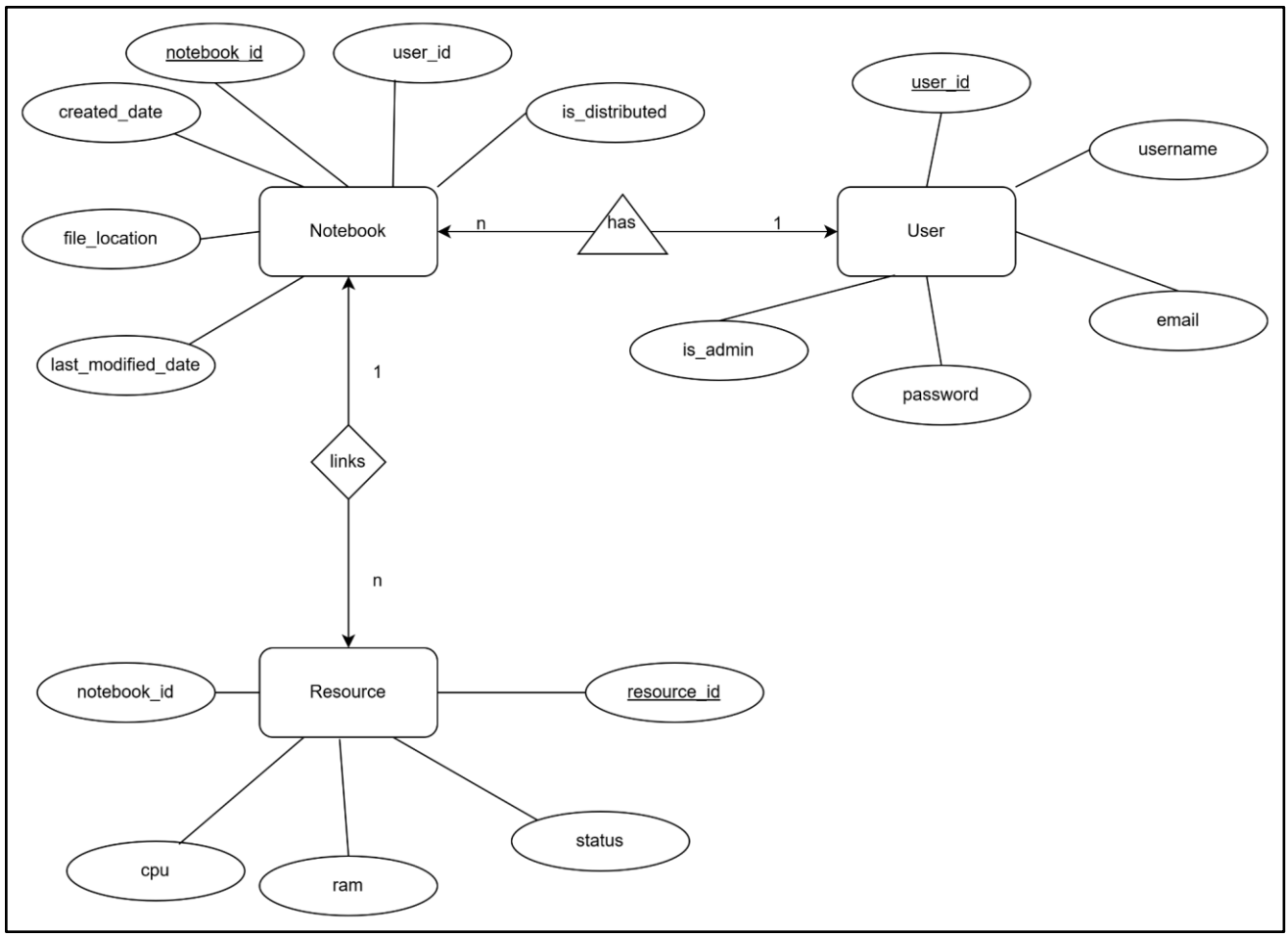


Figure-8: ER-Diagram of FleetTrain

Since the user and notebook have one to many relationships, the primary key of the user should be replicated in the notebook table. Same is applicable for notebook and resource.

The schemas derived from the ER-diagram is as follows:

Entity	Attribute	Type
User	user_id (PK)	number
	username	varchar(250)
	email	varchar(250)
	password	varchar(250)
	is_admin	boolean
Notebook	notebook_id (PK)	number
	created_date	date
	last_modified_date	date
	file_location	varchar(250)
	is_distributed	boolean
	user_id (FK)	number
Resource	resource_id (PK)	number
	status	varchar(250)
	cpu	varchar(250)
	ram	number
	notebook_id (FK)	number

Table-1: Schema of FleetTrain

Class-Based Modeling

Class-based modeling is a visual representation in software engineering that illustrates the structure of a software system using classes, their attributes, and relationships. It is commonly used in object-oriented design and modeling.

Analysis Classes

After identifying nouns from the scenario, I have filtered nouns belonging to the solution domain using General Classification (External entities, Things, Events, Roles, Organizational units, Places, and Structures). Nouns selected as a potential class were filtered using Selection Criteria (Retained information, Needed services, Multiple attributes, Common attributes, Common operations, and Essential requirements). After performing analysis on potential classes, I have found the following analysis classes:

- Developer
- Admin
- Notebook
- Resource
- Dataset
- Orchestrator
- ResourceManager

Class Cards

Developer Class	
Attributes	**Methods**
-developerID -name -email -password -notebooks	+signin() +signup() +signout() +editProfile() +createNotebook() +configureNotebook() +deleteNotebook() +downloadNotebook() +editNotebook() +executeNotebook() +deleteNotebook() +uploadDataset) +viewResourceUsage()
Responsibilities	**Collaborators**
Perform account functions (auth).	
Manage their own notebooks.	Notebook
Manage their own datasets.	Dataset
Monitor task progress and execution results.	Resource

Table-2: Developer Class Card

Admin Class	
Attributes	**Methods**
-adminID -name -email -password	+signin() +signup() +signout() +editProfile() +addResource() +removeResource() +monitorResources()
Responsibilities	**Collaborators**
Perform account functions (auth).	
Ensure system stability and resource optimization.	Resource

Table-3: Admin Class Card

Notebook Class	
Attributes	**Methods**
-notebookID -title -ownerID -cells -fileLocation -executionMode -executionStatus	+addCell() +editCell() +runCell() +setExecutionMode() +saveNotebook()
Responsibilities	**Collaborators**
Enable developers to write, execute, and manage code.	
Auto save after execution.	

Table-4: Notebook Class Card

Resource Class	
Attributes	**Methods**
-resourceID -name -status -capacity -notebookID	+releaseResource() +monitorUsage() +updateStatus()
Responsibilities	**Collaborators**
Represent a computational resource in the system.	
Show resource usage and availability.	

Table-5: Resource Class Card

Dataset Class	
Attributes	**Methods**
-notebookID -name -size -type -location	+uploadDataset() +deleteDataset() +viewDatasetDetails()
Responsibilities	**Collaborators**
Store and manage datasets uploaded by developers.	

Table-6:Dataset Class Card

ResourceManager Class	
Attributes	**Methods**
-resources -activeNotebooks	+distributeResources()
Responsibilities	**Collaborators**
Assign resources to the notebook tasks.	Resource

Table-7: ResourceManager Class Card

Orchestrator Class	
Attributes	**Methods**
-notebookId -connectedResources -taskQueue -datasetInfo -status	+distributeDataset() +orchestrateConnectedResources() +collectResults() +aggregateModel()
Responsibilities	**Collaborators**
Distribute dataset after random splitting.	Dataset
Aggregate the results from all nodes.	
Orchestrate the distributed nodes and collect results.	Resource

Table-8: Orchestrator Class Card

Class Responsibility Collaboration (CRC)

The CRC diagram of the classes can is given below:

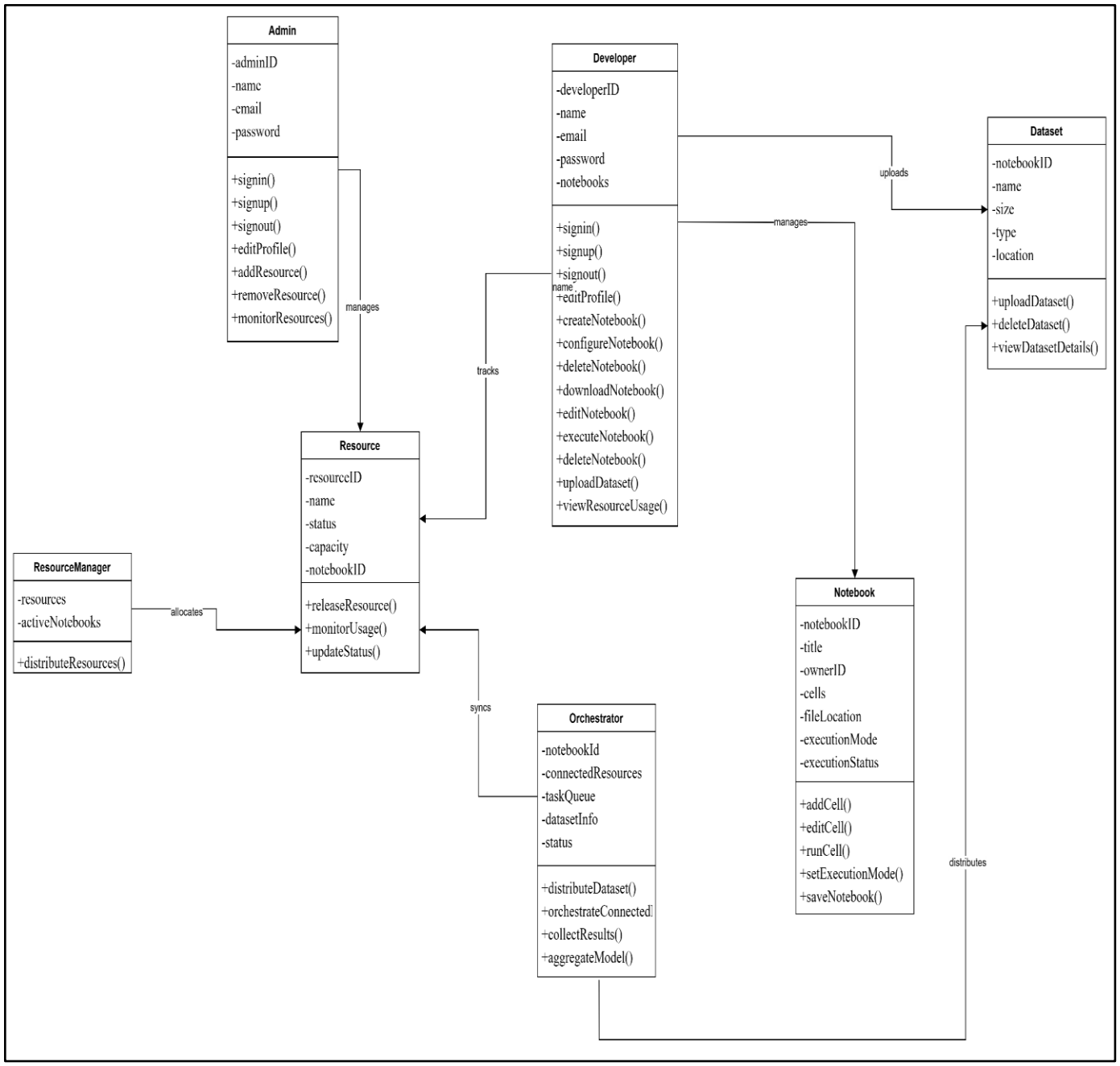


Figure-9: CRC-Diagram of FleetTrain

Architectural Design

Architectural design is a visual representation in software engineering that outlines the high-level structure and organization of a software system. It focuses on defining the major components or modules of the system, their interactions, and the overall system's architecture.

Architectural Context Diagram

The architectural context diagram for the tool is given below

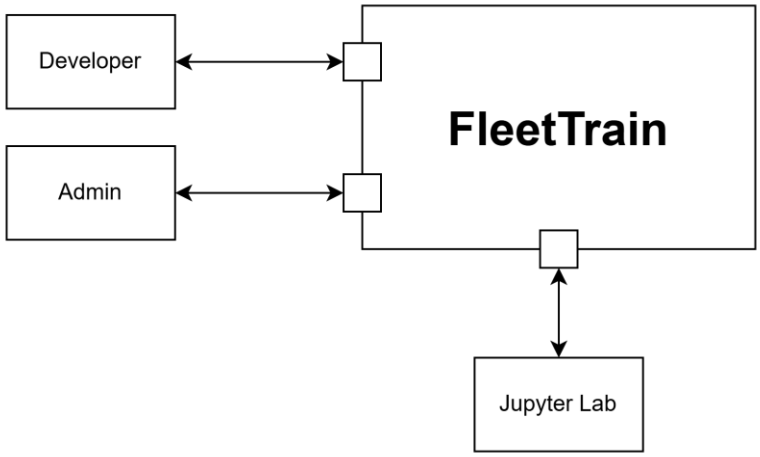


Figure-10: Architectural Context Diagram

Two categories of users are supposed to be the target for the tool. They are python developers and admin. Both types of users are the producers and also consumers of the information through the tool. There is one subordinate system, JupyterLab. The system will consume information(kernel instance information) from JupyterLab. However there are no superordinate or peer-level systems.

Top Level Components

Overall architectural structure with top level components is illustrated below

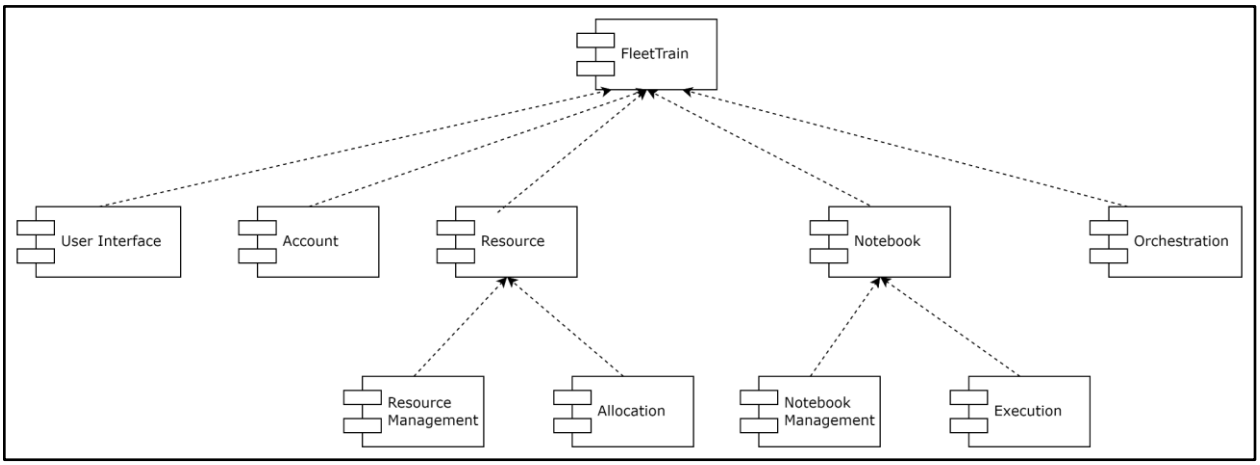


Figure-11: Top level components of FleetTrain

Define Archetypes

An archetype is a class or pattern that represents a core abstraction that is critical to the design of an architecture for the target system. In general, relatively small set archetypes are required to design even relatively complex systems. The target system architecture is composed of these archetypes, which represent stable elements of the architecture but may be instantiated many different ways based on the behavior of the system. In our cases the archetypes are:

- Model View Controller (MVC) Pattern
- Event Driven Architectural Pattern
- Client-Server Architecture
- Master-Worker Architectural Pattern

Model View Controller (MVC) Pattern:

The Model-View-Controller (MVC) pattern is an architectural design that separates an application into three interconnected components:

- Model: Represents the data, business logic, and rules of the application.
- View: Displays data to the user and handles the presentation layer.
- Controller: Acts as an intermediary, processing user input and updating the Model or View accordingly.

This separation enhances modularity, maintainability, and scalability, making it easier to manage and extend complex systems.

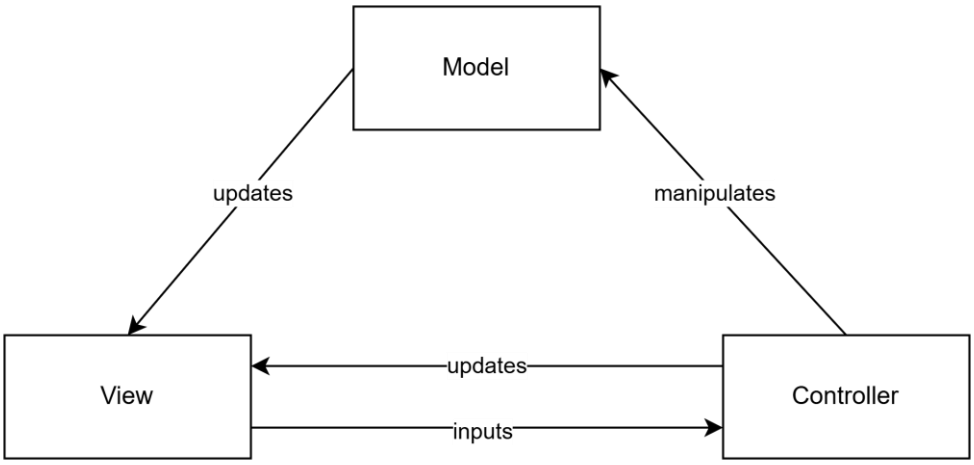


Figure-12: MVC Pattern

Event Driven Architectural Pattern:

Event-Driven Architectural Pattern is a design pattern where components communicate by producing and consuming events.

- Event Producers: Emit events whenever a change or action occurs.
- Event Consumers: Listen for and react to these events, performing tasks or triggering workflows.
- Event Brokers (optional): Facilitate event routing and decoupling between producers and consumers.

It is highly scalable, enables asynchronous processing, and is well-suited for dynamic, distributed, or real-time systems.

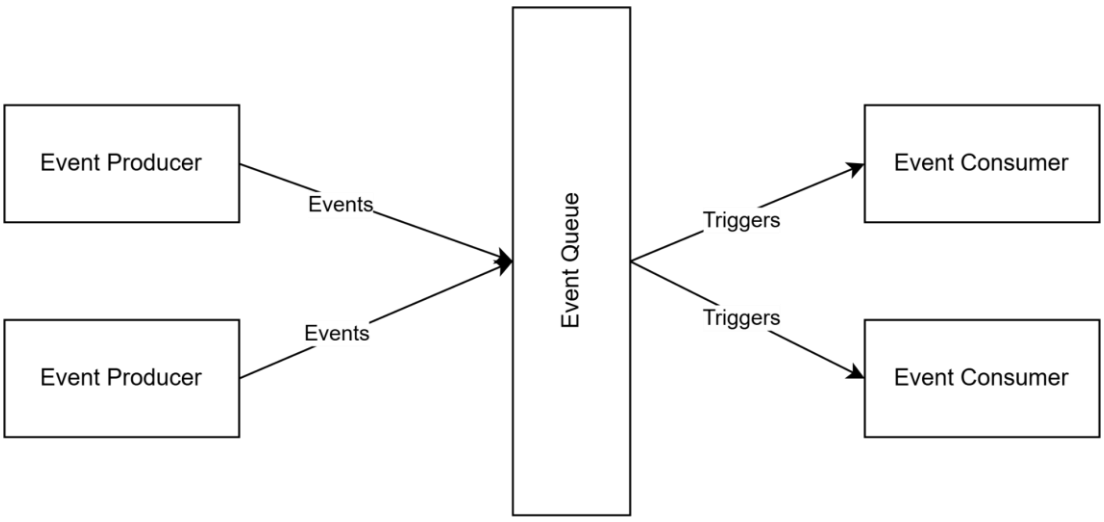


Figure-13: Event Driven Architecture

Client-Server Architecture:

Client-Server Architecture is a design pattern where tasks are divided between two entities:

- Client: Initiates requests and provides the user interface.
- Server: Processes requests, manages resources, and returns responses.

This architecture is widely used for networked systems, enabling centralized resource management and scalability while allowing multiple clients to interact with the server.

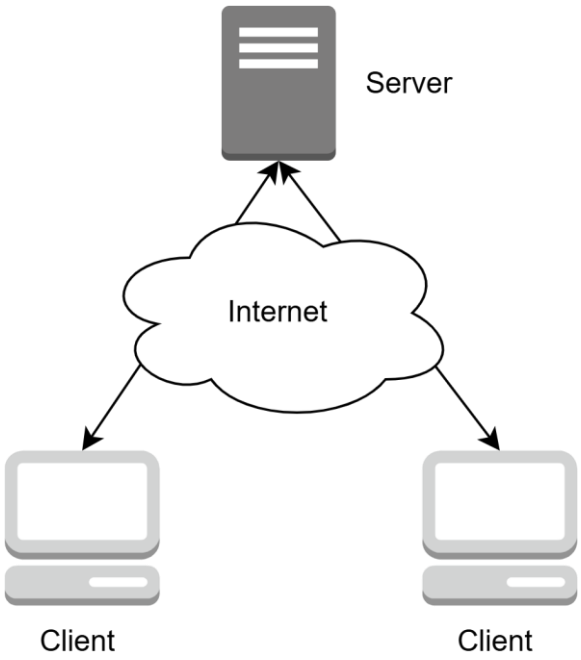


Figure-14: Client Server Architecture

Master-Worker Architectural Pattern:

Master-Worker Architectural Pattern is a design pattern where a central Master node delegates tasks to multiple Worker nodes.

- Master: Manages task distribution, monitors progress, and aggregates results.
- Workers: Execute tasks independently, reporting back to the Master once complete.

This architecture is ideal for parallel processing, workload distribution, and systems requiring centralized coordination with decentralized execution.

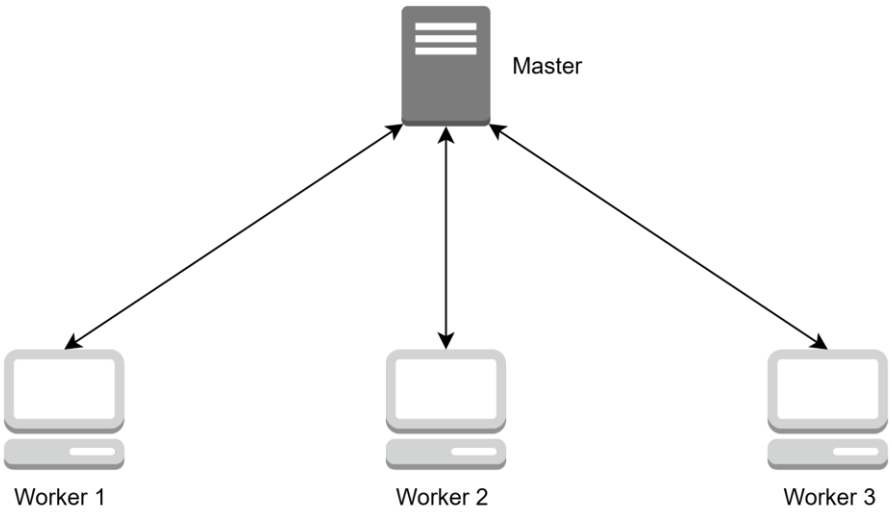


Figure-15: Master Worker Architecture

Refine the Architecture into Components

The archetypes that each of the components follow are given below:

Archetype	Component
MVC Pattern	User Interface, Account, Resource Management, Notebook Management
Event Driven Architectural Pattern	Resource Allocation
Client-Server Architecture	Notebook Execution
Master Worker Architectural Pattern	Orchestration

Table-9: Architecture to Component Refinement

Instantiations of the system

Notebook Management with MVC Pattern

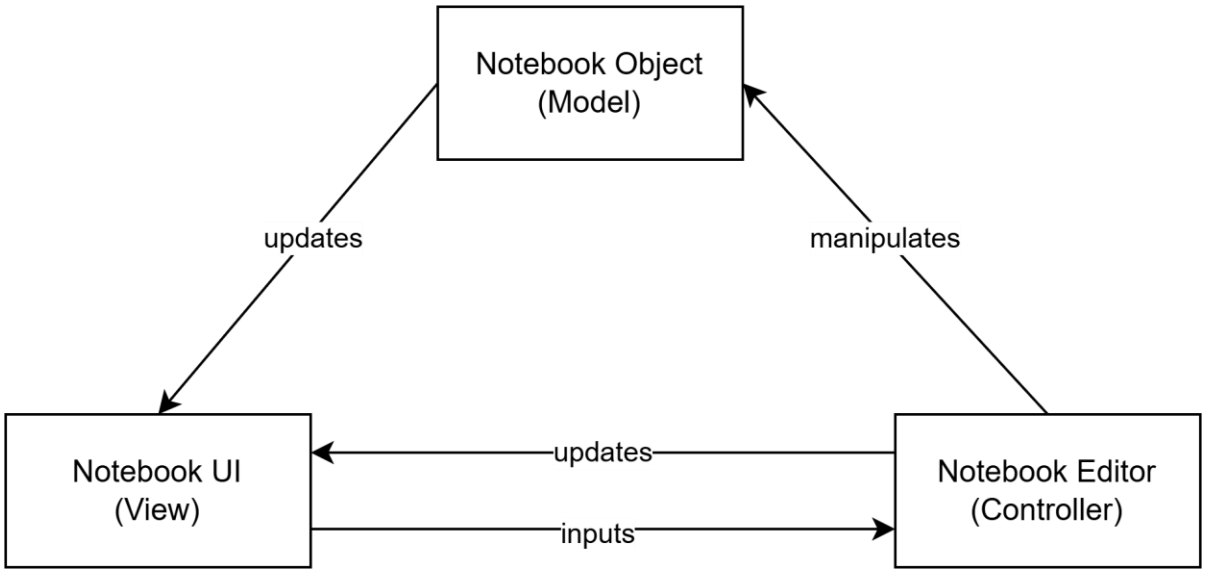


Figure-16: Notebook Management with MVC Pattern

Resource Allocation with Event Driven Architectural Pattern:

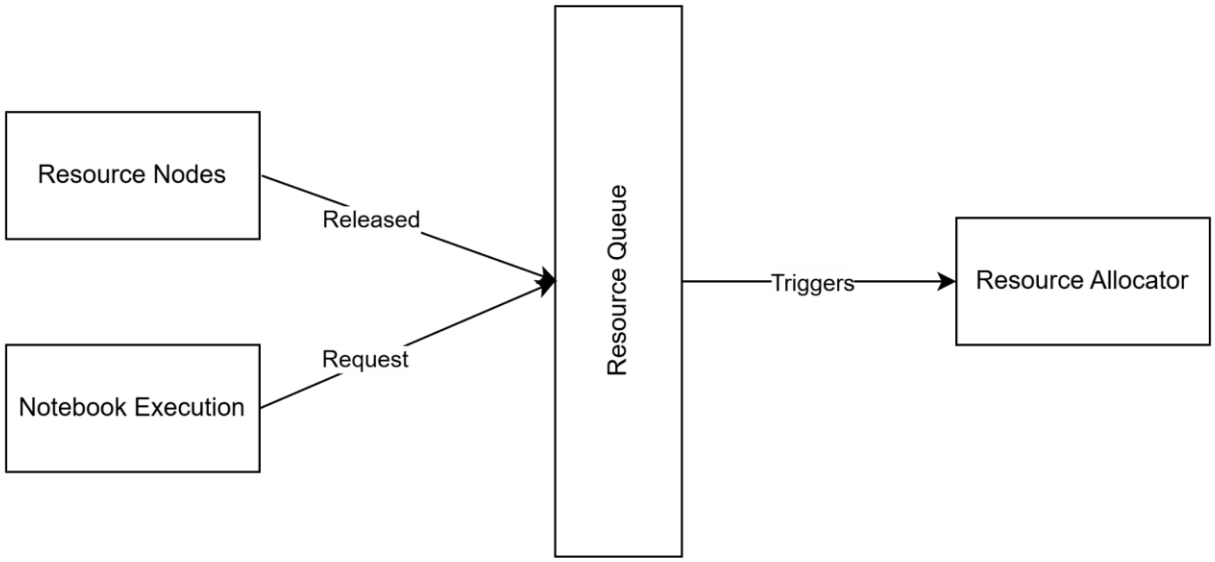


Figure-17: Resource Allocation with Event Driven Architectural Pattern

Notebook Execution with Client-Server Architecture:

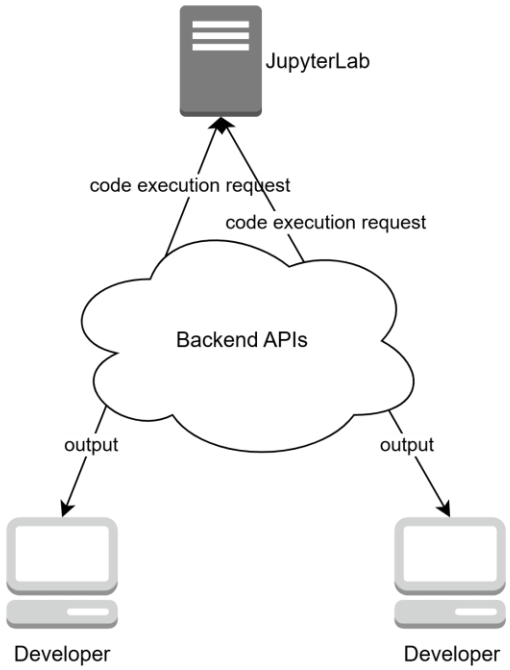


Figure-18: Notebook Execution with Client-Server Architecture

Orchestration with Master Worker Architectural Pattern:

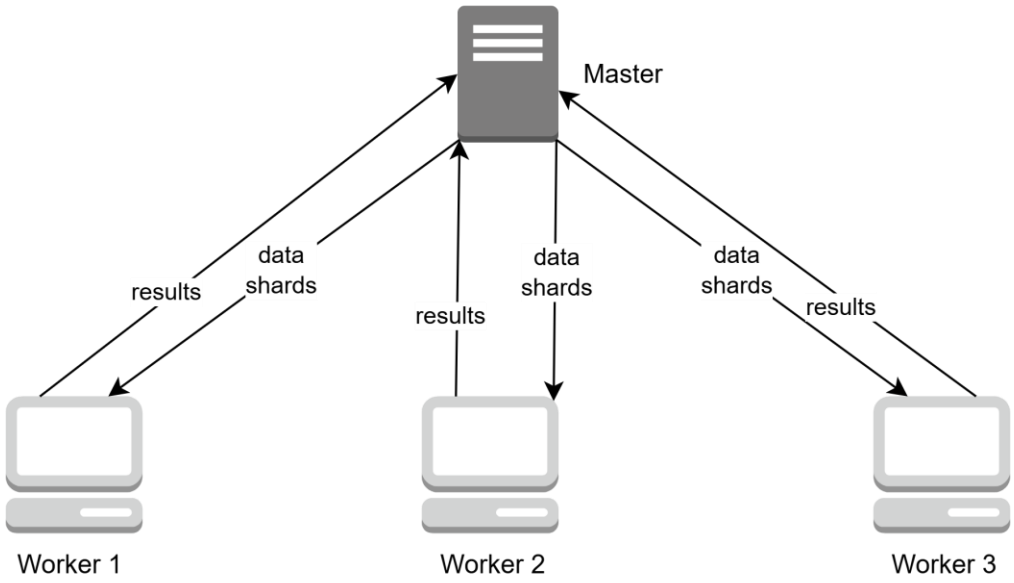


Figure-19: Orchestration with Master Worker Architectural Pattern

Deployment Diagram

Deployment Diagram of FleetTrain:

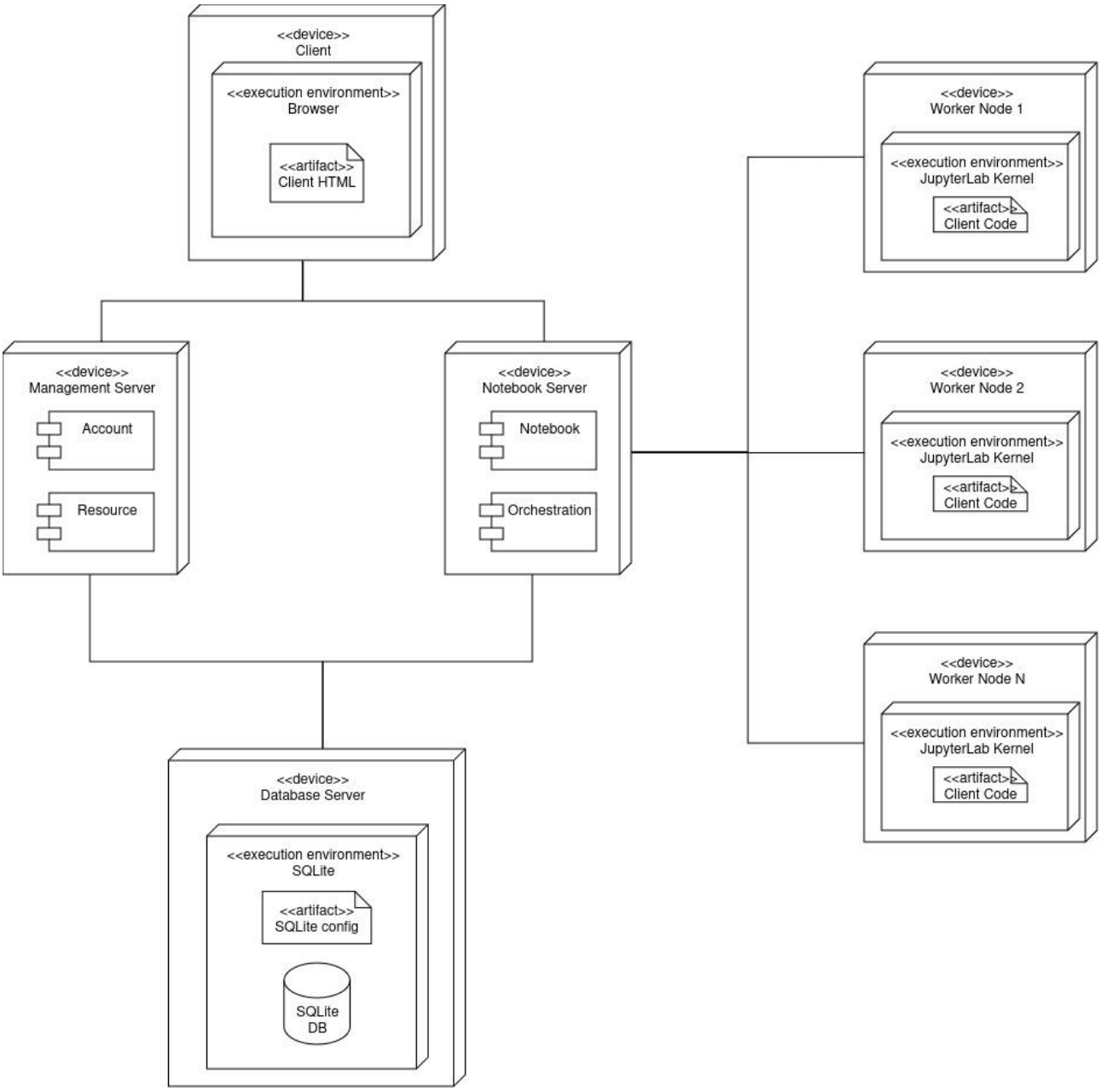


Figure-20: Deployment Diagram for FleetTrain

Preliminary Test Plan

In this chapter, a high level description of testing goals and summary of features to be tested are presented.

High Level Description of Testing Goals

Testing goals for this tool is represented below from a high level-

- To demonstrate that the tool meets its requirements
- To ensure the features of npm packages are are extracted correctly
- To ensure that it produces correct detection most of the time
- To ensure the false positive and negative results are mitigated
- To ensure that it provides detailed analysis report
- To ensure that the tool receives feedback from the users properly
- To ensure that the tool runs smoothly

Test Cases

Test Case 1: User Account Creation

Test Scenario:

- Create a new user account using a valid email and password.
- Submit the account creation form.

Expected Outcome:

- The user account is created successfully.
- A confirmation email is sent to the provided email address.

Test Case 2: User Sign-in and Sign-out

Test Scenario:

- Sign in using valid email and password.
- Sign out after successfully signing in.

Expected Outcome:

- User is signed in successfully.
- User is logged out successfully, and the login screen is displayed again.

Test Case 3: Notebook Creation and Execution

Test Scenario:

- After signing in, create a new notebook.
- Add cells to the notebook and execute them.

Expected Outcome:

- A new notebook is created without errors.
- Cells are executed successfully with appropriate output.

Test Case 4: Notebook Distributed Execution

Test Scenario:

- In an open notebook, set up one cell for distributed execution.
- Execute the notebook in a federated learning environment.

Expected Outcome:

- The notebook executes without errors.
- The federated learning tasks are distributed to the available nodes, and the global model is updated after aggregation.

Test Case 5: Resource Addition and Monitoring

Test Scenario:

- Admin adds a new computing resource (e.g., device or node).
- Monitor the usage of the new resource during notebook execution.

Expected Outcome:

- The new resource is successfully added.
- The resource's usage is tracked and displayed accurately in the admin panel.

Test Case 6: Notebook Deletion and Download

Test Scenario:

- After completing work, delete a notebook.
- Download a notebook to the local system before deletion.

Expected Outcome:

- The notebook is successfully deleted.
- The notebook can be downloaded in a supported format before deletion.

Test Case 7: Node failure handling

Test Scenario:

- During distributed execution of a notebook, one of the worker nodes crashes or fails due to some reason.

Expected Outcome:

- The orchestrator handles the error and allocates a new resource with the same notebook and gives that node the task of the failed node.
- The notebook execution continues.

Conclusion

In this report, I have discussed all the necessary technical details to develop the “FleetTrain”. I have discussed the scope, assumptions, QFD and usage scenario. Scenario-based modeling, data modeling and class-based modeling have been performed. The architectural design has been discussed. I have also presented the high-level description of testing goals and summary of items and features to be tested. I have added figures and tables for better understanding of the technical details.

Though FleetTrain addresses the automation in the federated machine learning, there are chances of further improvements. Addition of features like datathon competitions, community forum, dataset and model sharing can be the future scope of FleetTrain.

With this document, I have tried to clearly state the workflow of the total system. I hope this report can help the reader have an appropriate understanding of the technical details of this tool.

References

1. Qi, P., Chiaro, D., Guzzo, A., Ianni, M., Fortino, G., & Piccialli, F. (2023). Model aggregation techniques in federated learning: A comprehensive survey. *Future Generation Computer Systems*, *150*, 272–293. https://doi.org/10.1016/j.future.2023.09.008
2. Wang, Y., Koulouzis, S., Bianchi, R., Li, N., Shi, Y., Timmermans, J., Kissling, W. D., & Zhao, Z. (2022). Scaling notebooks as re-configurable cloud workflows. Data Intelligence, 4(2), 409–425. https://doi.org/10.1162/dint_a_00140
3. Debauche, O., Mahmoudi, S. A., Mahmoudi, S., & Manneback, P. (2018). Cloud Platform using Big Data and HPC Technologies for Distributed and Parallels Treatments. Procedia Computer Science, 141, 112–118. https://doi.org/10.1016/j.procs.2018.10.156